Workload- and Memory-Aware Scheduler for Heterogeneous Embedded MCU Cores

Jason Tang (jasont2) Catherine Li (catheri5)

November 9, 2026

Project URL: https://jasontxfei.github.io/15418-mcu-scheduler/

1 Summary

We plan to build a lightweight task-runtime library for the PSoC 6 family of multi-core embedded microcontrollers, initially targeting the CY8CPROTO-063-BLE board, comparing runtime and costs of weighted static assignment, work stealing, and cost-aware scheduling. We will implement an initial profiling phase estimating per-core execution and communication costs, and the scheduler combines these estimates with core availability, core capabilities, and task dependencies to decide how to distribute tasks.

2 Background

Heterogeneous embedded microcontrollers combine processors with different capabilities in a shared-memory system. The dual-core PSoC 6 contains a Cortex-M4 and Cortex-M0+ that can execute separate firmware while accessing shared SRAM and peripherals. Its builtin inter-processor communication (IPC) hardware provides locking and notification mechanisms for coordinating the cores. A common design approach assigns each core a fixed role (e.g. using one core for data acquisition and communication with sensors and the other for computation and processing). The manual assignment offers predictable behavior and low scheduling overhead, but may leave a core idle when task costs or input rates vary. Because the cores have different capabilities, a balanced assignment also depends on the particular operations being executed rather than simply halving the number of jobs.

Our runtime will represent application work as jobs with explicit dependencies. The application registers supported operations and submits an op identifier, input and output buffers, input sizes, and predecessor jobs. Each core executes its locally compiled implementation of the registered operation. Once a job's dependencies have been completed, the runtime assigns ready jobs according to one of the following policies:

Policy	Assignment strategy
Equal static assignment	Assign equal number of jobs to each core regardless of each job's specific execution time
Weighted static assignment	Assign jobs before execution using initial profiling measurements, determining a fixed allocation before execution
Work stealing	Allow an idle core to take unstarted, ready jobs from the other core's task queue.
Cost-aware work stealing (extension of work stealing)	Estimate execution and coordination costs, and redistribute job to available core only if estimated completion time improves.

Table 1: Scheduling policies evaluated by the runtime.

We will evaluate the runtime using controlled task graphs and a multichannel FIR-filtering workload. Generated sample blocks allow us to vary task size, filter length, dependency structure, and memory traffic (by playing with knobs such as samples per job, number of channels/parallel inputs, number of taps in the filter) while checking results against a serial reference.

3 The Challenge

Embedded workloads typically vary in available parallelism, with different computation amounts, memory traffic, and variable input-arrival timing (jitter). These are all factors that will affect the benefit of dynamic scheduling (or lack thereof) over a tuned static assignment.

3.1 Workload

- **Granularity and Memory Traffic:** Small jobs may not justify dispatch and synchronization overhead. Filtering and synthetic jobs will vary computation per sample, buffer traffic, and job size.
- **Dependencies:** Separate input channels can execute processing independently, while jobs sharing filter state (e.g. successive blocks from the same channel keeping a running average) must preserve ordering. Dependency chains restrict the number of jobs that can run concurrently.
- **Arrival Rate and Jitter:** Even with periodic sampling, buffering and interrupt-service delays can make sample blocks available irregularly. Bursts increase queue occupancy and latency, while gaps may leave workers idle.

3.2 Constraints

The M4 and M0+ have unequal processing capabilities but share SRAM and interconnect resources. Concurrent execution may therefore introduce contention, while queues, buffers, and dependency metadata compete for limited RAM (288 KB on the CY8CPROTO-063-BLE). We will reserve a configurable shared-memory budget for runtime queues and dependency metadata rather than assuming all on-chip RAM is available; application buffers and per-core stacks must also fit within the overall memory allocation. Processing must also keep pace with incoming blocks to avoid buffer exhaustion. Our multi-board stretch goal would also add SPI or UART transfer and synchronization costs, as they have low communication bandwidth.

3.3 Cost Model

Initial profiling will estimate per-core execution and coordination costs for each operation and input size. Contention and changing core availability can reduce prediction accuracy, making the best assignment difficult to determine from isolated measurements.

3.4 Scheduling Algorithm

Weighted static assignment minimizes runtime decision overhead but cannot correct unexpected imbalance. Ordinary work stealing redistributes ready jobs, while cost-aware stealing additionally considers predicted completion times. We will need to come up with the algorithms for deciding how much weight to assign each factor (core availability, execution cost, etc.) as well as for estimating completion times. We will also evaluate whether improved placement justifies the additional scheduling cost.

4 Resources

We will use the CY8CPROTO-063-BLE dual-core PSoC 6 board, C/C++, and hardware timers for profiling. Infineon's board support package provides board initialization support [**?**]. We will implement the scheduling runtime ourselves.

For programming background, we will consult Infineon's getting-started guide for PSoC 6 on ModusToolbox, its general code-example collection, and Cypress's introductory PSoC 6 101 video series. Specific peripheral and IPC examples will be selected during implementation.

For scheduling, we will consult Blumofe and Leiserson's work-stealing paper for dynamic load balancing, and Topcuoglu et al.'s HEFT paper for cost-based assignment of dependent tasks to heterogeneous processors. These will serve as algorithmic references rather than commitments to reproduce the complete systems.

5 Goals and Deliverables

5.1 Plan to Achieve

- **Runtime:** Implement the policies in Table **??**, shared-memory queues, completion tracking, and task dependencies.
- **Profiling:** Measure per-core operation costs and coordination overhead to guide weighted static and cost-aware assignment.
- **Benchmarks:** Use multichannel filtering and synthetic task graphs to vary job size, computational intensity, dependencies, imbalance, input arrival rates/jitter, and queue/buffer capacity.
- **Evaluation:** Validate outputs against a serial reference and check job completion and dependency ordering. Measure execution time, throughput, idle time, scheduling overhead, memory usage, and streaming latency.
- **Analysis:** Compare identical workloads across policies and M4-only execution, repeating measurements to assess variability. Identify when dynamic scheduling outperforms equal and weighted static assignment and when its overhead outweighs the benefit.

5.2 Hope to Achieve

Extend to two PSoC 6 boards over SPI or UART, evaluating when remote assignment and batched transfers justify communication overhead.

5.3 Fallback Goals

Evaluate independent jobs with fixed calibration before adding dependency graphs, prioritizing correctness and scheduling analysis.

6 Platform Choice

We will use the CY8CPROTO-063-BLE board, which provides heterogeneous Cortex-M4 and Cortex-M0+ cores, shared SRAM, and hardware IPC. Their different processing capabilities make equal work distribution potentially inefficient, while shared-memory contention and coordination costs may limit the benefit of activating both cores.

The board's 288 KB of SRAM must accommodate firmware data, per-core stacks, application buffers, and runtime metadata. A configurable runtime memory budget allows us to investigate scheduling under different resource limits. Firmware control over inter-core coordination also enables direct measurement of dispatch, synchronization, and idle time, supporting a study of when scheduling mechanisms justify their cost on constrained embedded hardware.

7 Schedule

Date	Deliverable
Nov. 9–15	Set up both cores; establish M4-only benchmarks and initial per-core timing measurements.
Nov. 16–22	Implement shared-memory queues, completion/dependency tracking, and equal and weighted static assignment.
Nov. 23	Submit milestone report with preliminary measurements and an updated schedule.
Nov. 23–29	Implement ordinary and cost-aware work stealing; validate correctness across policies.
Nov. 30–Dec. 6	Evaluate task sizes, computation intensity, memory budgets, and arrival jitter; profile and optimize bottlenecks.
Dec. 7–10	Complete experiments, figures, and final report. Submit the final report by Dec. 10.
Dec. 11	Present the project and evaluation at the poster session.

Stretch work: If core deliverables are complete, investigate two-board communication and batched remote job assignment.

Key deadlines from the proposal guidelines:

- Proposal: November 9, 2026, 11:59 p.m.
- Milestone report: November 23, 2026, 11:59 p.m.
- Final report: December 10, 2026, 11:59 p.m.
- Poster session: December 11, 2026.

References

References

[1] Infineon Technologies. *AN215656: PSoC 6 MCU Dual-Core System Design*. https://documentation.infineon.com/psoc6/docs/yvt1667482437523/index.html.

[2] Infineon Technologies. *PSoC 6 Peripheral Driver Library: IPC (Inter Process Communication)*. https://infineon.github.io/psoc6pdl/pdl_api_reference_manual/html/group__group__ipc.html.

[3] Infineon Technologies. *CY8CPROTO-063-BLE Board Support Package*. https://github.com/Infineon/TARGET_CY8CPROTO-063-BLE.

[4] Infineon Technologies. *AN228571: Getting Started with PSoC 6 MCU on ModusToolbox Software*. https://documentation.infineon.com/psoc6/docs/pvw1667470959605.

[5] Infineon Technologies. *Code Examples for ModusToolbox Software*. https://github.com/Infineon/Code-Examples-for-ModusToolbox-Software.

[6] Cypress Semiconductor. *PSoC 6 101*. Video tutorial series. https://www.youtube.com/playlist?list=PLIOkqhZiy83H81AhLcjwIKkB9w07JQZS6.

[7] Robert D. Blumofe and Charles E. Leiserson. "Scheduling Multithreaded Computations by Work Stealing." *Journal of the ACM*, 46(5):720–748, 1999. https://doi.org/10.1145/324133.324234.

[8] Haluk Topcuoglu, Salim Hariri, and Min-You Wu. "Performance-Effective and Low-Complexity Task Scheduling for Heterogeneous Computing." *IEEE Transactions on Parallel and Distributed Systems*, 13(3):260–274, 2002. https://doi.org/10.1109/71.993206.